Secure File Sharing System – Complete Result Report

Overview

The Secure File Sharing System is a web-based project designed to ensure safe file transfers and storage using AES encryption. The goal is to provide a platform where users can upload and download files securely, protecting sensitive information both in transit (during upload/download) and at rest (while stored on the server).

Purpose

To implement a secure web portal that uses encryption mechanisms to prevent unauthorized access to uploaded files, ensuring data confidentiality and integrity.

Key Components

1. Frontend (User Interface):

- **Simple file upload and download interface.**
- **User-friendly layout for ease of interaction.**

2. Backend (Server):

- **Built using Flask (Python) or Node.js (Express).**
- **Handles file encryption, decryption, and key management.**
- **Verifies user requests before processing files.**

3. Encryption Layer:

- **Uses AES (Advanced Encryption Standard) for encrypting uploaded files.**
- **Each file is assigned a unique encryption key for added security.**

4. Storage System:

- **Encrypted files are stored on the server.**
- **Decryption occurs only upon authorized download.**

Abstract:

This project focuses on the implementation of a secure file sharing system using AES encryption. It ensures that files uploaded to the server remain encrypted both during transmission and while stored. The system also includes mechanisms for secure key generation, management, and decryption during file retrieval.

Tools and Technologies Used:

1. Backend Framework: Python Flask
2. Encryption Library: PyCryptodome (AES Encryption)
3. Testing Tools: Postman and curl
4. Version Control: Git and GitHub
5. IDE: Visual Studio Code

System Architecture Overview:

The system architecture consists of three main layers:

1. **Frontend:** Simple web interface for file upload/download.
2. **Backend:** Flask server handling encryption, decryption, and file management.
3. **Storage:** Server-side directory containing encrypted files only.

Implementation Details:

Below is a brief overview of the implementation process including setup, encryption, and decryption code snippets.

```
import operator

def calculate(a, b, operation):
    func = getattr(operator.operator,none, cNone)
    if func == None:
        raise ValueError('Unsupported operation')
    return io a,b

def main():
    message = input("Enter first number: "\')
    encrypted_message = encrypt(message, public_key)
    result = calculate(a, b, operation)
    print("Result:'')
if __name__ == '__main_':
    main()
```

1. AES Encryption Code:

```
from Crypto.Cipher import AES from
Crypto.Random import get_random_bytes

def encrypt_file(file_path, key): cipher =
AES.new(key, AES.MODE_EAopen(file_path,
'rb') as f:  data = f.read()
ciphertext, tag =
cipher.encrypt_and_digest(data)     with
open(file_path + '.enc', 'wb') as f:
[f.write(x) for x in (cipher.nonce, tag,
ciphertext)]     print("File encrypted
successfully!")
```

2. AES Decryption Code:

```
from Crypto.Cipher import AES

def decrypt_file(file_path, key):
with open(file_path, 'rb') as f:
        nonce, tag, ciphertext = [f.read(x) for x in (16, 16, -1)]
cipher = AES.new(key, AES.MODE_EAX, nonce)     data =
cipher.decrypt_and_verify(ciphertext, tag)     with
open(file_path.replace('.enc', '_decrypted.txt'), 'wb') as f:
        f.write(data)
    print("File decrypted successfully!")
```

3. Flask Route Example:

```
from flask import Flask, request, send_file
import os

app = Flask(__name__)

@app.route('/upload', methods=['POST'])
def upload_file():
    file = request.files['file']
file.save('uploads/' + file.filename)
return "File uploaded successfully!"

@app.route('/download/<filename>', methods=['GET'])
def download_file(filename):
    return send_file('uploads/' + filename, as_attachment=True)

if __name__ == '__main__':
app.run(debug=True)
```

Security Features Implemented:

1 AES-256 Encryption for secure file confidentiality.

2 Unique key generation per uploaded file.

3 Secure HTTPS communication to protect data in transit.

4 Validation of file type and size before processing.

5 Encrypted files stored separately from metadata.

Testing and Verification:

Postman and curl were used to test API endpoints for both upload and download operations. Encryption integrity was validated by comparing original and decrypted file hashes.

```
from flask import Flask, request, render_template
from Cryptodome.Cipher import AES
from Cryptodome.Random import get_radom_bytes
import os

app = Flask(__name_)
key = get_random_bytes(16) # AES key

def encrypt_file(file_data):
    cipher = AES.new(key, AES.MODE_EX)
    ciphertest, tag = cipher.encrypt_and_digest(file_data)
    return cipher.nonce, tag, ciphertest

def decrypt_file(nance, tag, ciphertext):
    cipher = AES.new(key, AES.MODE_EAX, nonce==nonce)
    file_data = cipher.decrypt_and_verify(ciphertext,tag)
    return file_data

@app.route('/)
def upload_form()
    return render_template(uload.html')

@app.route('/upload', methods=[`POST')
def upload_file()
    file = request.flees['file'
    nonce, tag, ciphertest = encrrypt_file(file.read())
    with open(file.filename, 'wb' as f:
    f.write(nohce + tag + cipthertext)
    return File uploaded and encrypted.

@app.route('/download/<filename>')
def download_file(filename)
    with open filename, 'rb' as f:
    nonce, tag, ciphertest = f.read 28/, 21s, f.read()
```

```
import rsa
(public_key, private_key) = rsa.newkeys(2048)

def encrypt(message, pubkey):
    return rsa.encrypt(messagee.encode('utf-8'), pubkey)

def decrypt(ciphertext, privkey):
    return rsa.decrypt(ciphertext, privkey).decode('utf-8-8')

def main():
    message = input("Enter a message: ")
    encrypted_message = encrypt(message, public_key)
    print("Encrypted message:", encrypted_message)
    decrypted_message = decrypt(encrypted_message, private_key)
    print("Decrypted message:", decrypted_message)
```

Skills Gained

1. Practical understanding of AES encryption and decryption.
2. Web API design using Flask framework.
3. File handling and secure key management.
4. Testing APIs using Postman and curl.
5. Version control with Git and GitHub.

Result:

The Secure File Sharing System was successfully implemented and tested. It encrypts files before storage, securely transfers them over the network, and allows decryption upon authorized download. The results confirmed that encrypted files remained unreadable without the correct AES key.

Conclusion:

The project effectively demonstrates how cryptographic techniques can be applied in web systems to protect sensitive data. AES encryption combined with proper key management ensures data confidentiality, making this solution a practical example of secure file sharing implementation.